

## Contenido

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| UT1 Arquitectura Web, protocolos y herramientas - PROVISIONAL.....                  | 2  |
| 1 Introducción.....                                                                 | 2  |
| 2 Conceptos generales de arquitectura web .....                                     | 2  |
| 3 Aplicaciones Web .....                                                            | 2  |
| 3.1 ¿Qué es la web? .....                                                           | 2  |
| 3.2 ¿Qué son los protocolos como HTTP/HTTPS? .....                                  | 2  |
| 3.3 Arquitectura cliente-servidor web .....                                         | 5  |
| 3.4 Página web.....                                                                 | 6  |
| 3.5 Sitio web .....                                                                 | 6  |
| 3.6 Aplicación web.....                                                             | 6  |
| 3.6.1 Beneficios de las aplicaciones web .....                                      | 6  |
| 3.6.2 Procesamiento de páginas en el lado del servidor .....                        | 7  |
| 3.6.3 Páginas web estáticas en el servidor .....                                    | 7  |
| 3.6.4 Páginas dinámicas en el servidor .....                                        | 9  |
| 3.6.5 Páginas dinámicas con acceso a una base de datos.....                         | 10 |
| 3.6.6 Cliente estático o cliente dinámico.....                                      | 12 |
| 3.6.7 Cliente estático creado a partir de texto plano (Static Site Generator) ..... | 13 |
| 4 Tecnologías de desarrollo Web .....                                               | 14 |
| 4.1 Integración de tecnologías de desarrollo .....                                  | 14 |
| 4.2 Sistemas gestores de contenido (CMS) .....                                      | 15 |
| 4.2.1 Headless CMS .....                                                            | 16 |

# UT1 Arquitectura Web, protocolos y herramientas - PROVISIONAL

## 1 Introducción

En este tema trataremos los conceptos generales de una aplicación web, así como las tendencias actuales relativas a las tecnologías más usadas en el desarrollo de aplicaciones web dinámicas.

## 2 Conceptos generales de arquitectura web

En esta unidad aprenderemos los siguientes conceptos.

1. Qué es una aplicación web.
2. Qué tecnologías se pueden utilizar para crear una aplicación web.
3. Qué es un servidor web.
4. Y, finalmente, en qué consiste el despliegue de una aplicación Web.

## 3 Aplicaciones Web

### 3.1 ¿Qué es la web?

La **Web (World Wide Web)** es un sistema distribuido de documentos y recursos interconectados mediante **hiperenlaces**, que se acceden a través de Internet utilizando protocolos como **HTTP/HTTPS**.

Con un navegador web, un usuario visualiza sitios web compuestos de páginas web que pueden contener textos, imágenes, vídeos u otros contenidos multimedia, y navega a través de esas páginas usando hiperenlaces. Por tanto un hiperenlace o hipervínculo es el elemento concreto que permite realizar esa navegación.

### 3.2 ¿Qué son los protocolos como HTTP/HTTPS?

#### ¿Qué es HTTP?

**HTTP (HyperText Transfer Protocol)** es un **protocolo de comunicación de la capa de aplicación** utilizado para el intercambio de recursos entre un **cliente** (normalmente un navegador) y un **servidor web**.

Normalmente utiliza el puerto **80**

Su funcionamiento básico sigue el modelo: **Cliente → Request → Servidor → Response → Cliente**

Por ejemplo, cuando escribimos <http://www.ejemplo.com/index.html> el navegador realiza una petición al servidor solicitando ese recurso.

El servidor procesa la petición y devuelve una respuesta, que puede contener HTML, CSS, JavaScript, imágenes, etc.

#### ¿Qué es HTTPS?

**HTTPS (HyperText Transfer Protocol Secure)** es HTTP utilizado sobre una conexión protegida mediante **TLS (Transport Layer Security)**.

Su objetivo principal es proporcionar:

- **Confidencialidad:** los datos viajan cifrados.
- **Integridad:** se puede detectar si los datos han sido modificados durante el transporte.
- **Autenticación del servidor:** mediante certificados digitales, el cliente puede verificar la identidad del servidor.

Debemos tener claro que HTTPS no es un protocolo completamente diferente de HTTP. Es HTTP funcionando sobre una conexión protegida mediante TLS. Podemos resumirlo como **HTTPS = HTTP + TLS**

Normalmente utiliza el puerto **443**.

### ¿Diferencias entre Hipertexto e hiperenlaces?

**Hipertexto** es el **contenido/documento** organizado de manera que permite navegar entre diferentes partes o documentos mediante enlaces.

**El hipertexto** es un sistema de información que permite relacionar documentos o partes de documentos mediante **hiperenlaces**. Un **hiperenlace** es un elemento interactivo que permite acceder a otro recurso al seleccionarlo.

Por ejemplo, si en una página web tenemos:

«Consulta la información sobre **redes de ordenadores**.»

Si «**redes de ordenadores**» aparece resaltado y al hacer clic nos lleva a otra página:

- La página completa forma parte del **hipertexto**.
- «redes de ordenadores» es el **hiperenlace**.
- La dirección a la que conduce es el **destino del enlace** (por ejemplo, una URL).

Por tanto:

- **Hipertexto:** forma de organizar la información permitiendo navegar entre diferentes recursos.
- **Hiperenlace:** elemento que permite acceder a otro recurso mediante un clic o acción.
- **URL:** identifica la ubicación de un recurso.
- **HTTP/HTTPS:** protocolos utilizados principalmente para solicitar y transferir esos recursos entre cliente y servidor.

### ¿Cómo transfieren HTTP/HTTPS los recursos entre cliente y servidor?

La comunicación HTTP se basa fundamentalmente en dos **mensajes**:

- **Request - petición**
- **Response - respuesta**

#### Request - petición

Es el mensaje que envía el cliente al servidor para solicitar una operación.

Por ejemplo:

**GET /index.html HTTP/1.1**

**Host: www.ejemplo.com**

El navegador está diciendo “Quiero obtener /index.html del servidor www.ejemplo.com”.

### **Response - respuesta**

Es el mensaje que devuelve el servidor al cliente.

Por ejemplo:

**HTTP/1.1 200 OK**

**Content-Type: text/html**

**<html>**

...

**</html>**

El servidor indica que la petición se ha procesado correctamente (200 OK) y proporciona el recurso solicitado.

HTTP utiliza los **métodos GET** y **POST** para indicar al servidor qué operación solicita el cliente.

**GET** Se utiliza principalmente para solicitar/obtener información.

Ejemplo:

**GET /productos.html HTTP/1.1**

También puede enviar parámetros en la URL:

**https://ejemplo.com/buscar?producto=teclado**

Aquí:

- **/buscar** sería el recurso solicitado.
- **producto=teclado** sería el parámetro enviado al servidor.

**POST** Se utiliza principalmente para enviar datos al servidor, por ejemplo los datos introducidos en un formulario.

**POST /login HTTP/1.1**

Los datos se incluyen normalmente en el cuerpo (body) de la petición, no en la URL **usuario=gustavo&password=1234** pero ambos casos producen el mismo resultado.

Que los datos se envíen en el body mediante POST no significa que estén cifrados. Para protegerlos frente a terceros se utilizará HTTPS.

Entonces cuando por ejemplo escribimos una URL en el navegador estamos haciendo un **GET REQUEST** y cuando pulsamos sobre un formulario hacemos un **POST REQUEST**

## Resumiendo

- **HTTP (HyperText Transfer Protocol)** es un protocolo de la capa de aplicación que permite la comunicación entre clientes y servidores web mediante un modelo de petición-respuesta (REQUEST o RESPONSE). El cliente realiza una petición utilizando métodos como GET o POST, y el servidor devuelve una respuesta con un código de estado y, normalmente, el recurso solicitado o el resultado de la operación.
- **HTTPS** es HTTP protegido mediante **TLS**, proporcionando cifrado, integridad de los datos y autenticación del servidor mediante certificados digitales.
- **HTML** es una estructura en hipertexto donde los hiperenlaces conectan los recursos y sobre la que se construyen páginas web. **HTTP/HTTPS** permite que el navegador los solicite y el servidor los entregue.

## 3.3 Arquitectura cliente-servidor web

La arquitectura cliente-servidor web es un modelo de comunicación en el que un cliente web, normalmente un navegador, solicita recursos o servicios a un servidor web mediante los protocolos HTTP/HTTPS, y el servidor procesa dichas peticiones y devuelve una respuesta al cliente.

Su funcionamiento básico es:

**Cliente → Request HTTP/HTTPS → Servidor → Response HTTP/HTTPS → Cliente**

Por ejemplo, cuando un usuario introduce una URL o pulsa un hiperenlace:

- El navegador (cliente) identifica la URL del recurso que se quiere consultar.
- El cliente realiza una petición HTTP/HTTPS (Request) al servidor.
- La petición puede utilizar diferentes métodos, como GET para solicitar información o POST para enviar datos.
- El servidor web recibe y procesa la petición.
- El servidor genera una respuesta HTTP/HTTPS (Response) con un código de estado y, normalmente, el recurso solicitado.
- El navegador recibe la respuesta e interpreta, por ejemplo, el HTML, CSS y JavaScript para mostrar la página al usuario.

Si resumimos toda la idea en una cadena quedaría de esta forma:

**Usuario → Navegador → URL/hiperenlace → HTTP/HTTPS → Request → Servidor web → Response → HTML → Página web**

Cliente y servidor no son necesariamente dos ordenadores diferentes en sentido físico. Son roles dentro de la comunicación. Un mismo equipo puede actuar como cliente en una comunicación y como servidor en otra; por ejemplo, cuando trabajamos con localhost, el navegador y el servidor web pueden estar ejecutándose en el mismo ordenador.

### 3.4 Página web

Una página web es un documento electrónico escrito en **HTML** (HyperText Markup Language). Las páginas web están enlazadas a través de hiperenlaces (links). Mediante un navegador un usuario puede navegar a través de la web siguiendo los hiperenlaces

Las páginas web enlazan contenidos de naturaleza heterogénea:

- Imágenes: JPG, GIF, PNG, ...
- Documentos: PDF, TXT, ...
- Audio: MP3, WAV, ...
- Vídeo: AVI, MPEG, ...

### 3.5 Sitio web

Un sitio web (o portal) es una colección de páginas web relacionadas entre sí que suelen compartir la primera parte de la dirección web (el dominio). Ejemplos:

- <https://www.ieselcaminas.org>: Sitio web del Instituto
- <https://es.wikipedia.org>: Sitio web de la Wikipedia en español

### 3.6 Aplicación web

Aplicación web son aquellas herramientas que los usuarios pueden utilizar accediendo a un servidor web a través de Internet/Intranet mediante un navegador. Es una aplicación software codificada en lenguaje soportado por los navegadores web en la que se confía la ejecución al navegador.

Las aplicaciones web son populares debido a lo práctico del navegador web como cliente ligero, a la independencia del sistema operativo, así como a la facilidad para actualizar y mantener aplicaciones web sin distribuir e instalar software a miles de usuarios potenciales.

Existen aplicaciones como los webmails, wikis, weblogs, tiendas en línea y la propia Wikipedia que son ejemplos bastante conocidos de aplicaciones web.

#### 3.6.1 Beneficios de las aplicaciones web

Las aplicaciones Web se ejecutan en múltiples plataformas, independientemente del sistema operativo o dispositivo, siempre y cuando el navegador sea compatible.

- Todos los usuarios acceden a la **misma versión**, eliminando cualquier problema de compatibilidad.
- **Reducen los costes** tanto para el negocio como para el usuario final, ya que hay menos soporte y mantenimiento requeridos por el negocio y menores requerimientos para el equipo del usuario final.
- **No requiere instalar software especial** (en los clientes ). En esencia, para acceder a un software web solo necesitamos disponer de un navegador de páginas web (Internet Explorer, Firefox, Opera, Chrome, etc.). No es necesario tener nada más. Debido a la arquitectura de las aplicaciones web, el navegador suele quedar relegado a mostrar la interfaz de usuario (menús, opciones,

formularios, etc.), mientras que toda la compleja lógica de negocio se lleva en el lado del servidor.

- **Información centralizada.** En una aplicación web, no solamente la lógica de negocio está centralizada en el servidor, sino también los datos que se ubican en una base de datos centralizada (en ese servidor u otro destinado a tal fin). La centralización tiene la ventaja de facilitar el acceso a la misma.
- **Seguridad y copias de seguridad.** Este es un corolario del punto anterior, es decir, una consecuencia. Como disponemos de los datos centralizados es más fácil establecer y llevar el control de una política de copias de seguridad centralizada.

### 3.6.2 Procesamiento de páginas en el lado del servidor

Desde el punto de vista del **servidor web**, una aplicación Web es un conjunto de páginas Web estáticas y dinámicas. Una página Web estática es aquella que no cambia cuando un usuario la solicita: el servidor Web envía la página al navegador Web solicitante sin modificarla. Por el contrario, el servidor modifica las páginas Web dinámicas antes de enviarlas al navegador solicitante. La naturaleza cambiante de este tipo de página es la que le da el nombre de dinámica.

Por ejemplo, podría diseñar una página para que mostrara los resultados de las notas de un alumno y dejara cierta información fuera (como el nombre del alumno y sus notas) para calcularla cuando la página la solicite un alumno en particular.

### 3.6.3 Páginas web estáticas en el servidor

Un sitio Web estático consta de un conjunto de páginas y de archivos HTML relacionados alojados en un equipo que ejecuta un servidor Web.

Un **servidor Web** es un software que suministra páginas Web en respuesta a las peticiones de los navegadores Web. La petición de una página se genera cuando el usuario hace clic en un vínculo de una página Web, elige un marcador en un navegador o introduce una URL en el cuadro de texto Dirección del navegador.

El contenido final de una página Web estática lo determina el diseñador de la página y no cambia cuando se solicita la página. A continuación se incluye un ejemplo:

```
<html>

<head lang="es" style="background-color: #F0F0F0; color: #000000; font-family:
Arial, sans-serif;">

<title>Implantación de arquitecturas Web</title>

</head>

<body lang="es" style="background-color: #F0F0F0; color: #000000; font-family:
Arial, sans-serif;">

<h1>Acerca del módulo</h1>

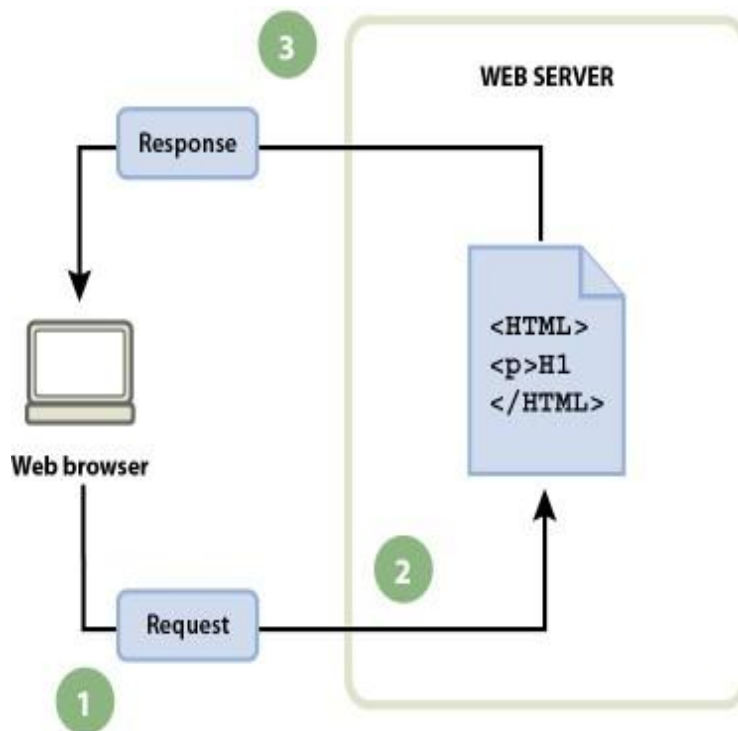
<p>Puesta en producción segura ...</p>

</body>

</html>
```

El diseñador escribe todas y cada una de las líneas de código HTML de la página antes de colocarla en el servidor. El código HTML no cambia una vez colocado en el servidor y por ello, este tipo de páginas se denomina página estática.

Cuando el servidor Web recibe una petición de una página estática, el servidor lee la solicitud, localiza la página y la envía al navegador solicitante, como se muestra en el siguiente ejemplo:



**Figura 2:** Página Estática

**1.** El navegador web solicita la página estática **2.** El servidor localiza la página **3.** El servidor Web envía la página al navegador solicitante. Finalmente, el navegador renderiza la página mostrando el siguiente resultado.

Este ejemplo está ejecutado desde el lado del Live Server. Desde Visual Code Estudio botón derecho sobre la página web y pulsamos Abrir con el Live Server. También podemos escribir en el cuadro de direcciones del navegador <http://127.0.0.1:5500/Carpeta/fichero.html>



**Figura 3:** Página estática. Ejemplo

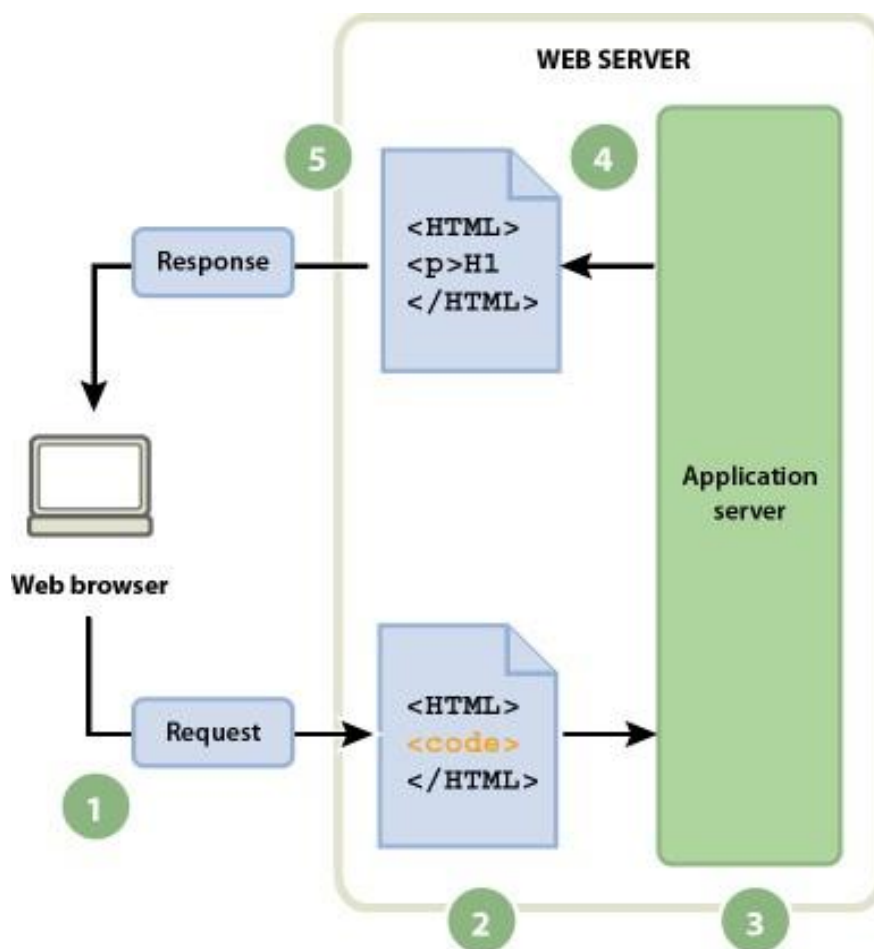
Actualmente esta arquitectura se usa principalmente para:

1. Páginas personales
2. Páginas de proyectos software
3. Documentación técnica ([JavaDoc](#) en Java, Maven site, etc...)

#### 3.6.4 Páginas dinámicas en el servidor

Cuando un servidor Web recibe una petición para mostrar una página Web estática, el servidor la envía directamente al navegador que la solicita, sin embargo, cuando el servidor Web recibe una petición para mostrar una página dinámica reacciona de distinta forma ya que transfiere la página a un software especial encargado de finalizar la página. Este software especial se denomina **servidor de aplicaciones**.

El servidor de aplicaciones lee el código de la página, finaliza la página en función de las instrucciones del código y elimina el código de la página. El resultado es una página estática que el servidor de aplicaciones devuelve al servidor Web, que a su vez la envía al navegador solicitante. Lo único que el navegador recibe cuando llega la página es código HTML puro. A continuación se incluye una vista de este proceso:



**Figura 4:** Página Dinámica

1. El navegador web solicita la página dinámica.
2. El servidor web localiza la página y la envía al servidor de aplicaciones.
3. El servidor de aplicaciones busca instrucciones en la

página y la termina. **4.** El servidor de aplicaciones pasa la página terminada al servidor web. **5.** El servidor web envía la página finalizada al navegador solicitante que la renderiza. Un ejemplo sería el siguiente código PHP.

```
<!DOCTYPE html>

<html>

<body>

<?php

$txt1 = "Learn PHP";

$txt2 = "W3Schools.com";

$x = 5; $y = 4; echo "<h2>" . $txt1 . "</h2>";

echo "Study PHP at " . $txt2 . "<br>";

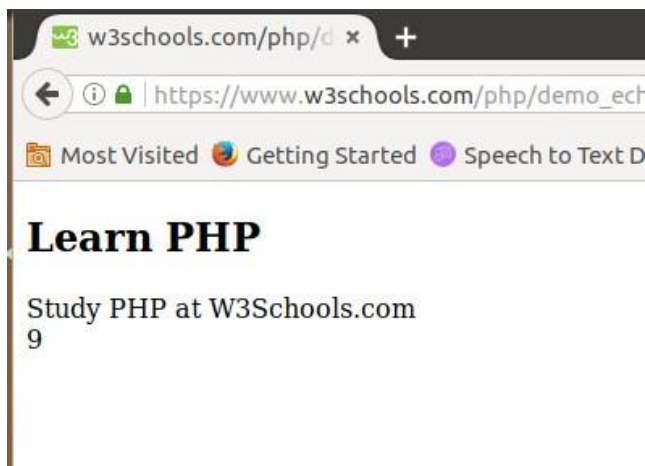
echo $x + $y;

?>

</body>

</html>
```

Y este sería el resultado mostrado en el navegador, donde el servidor de aplicaciones ha interpretado las órdenes PHP



**Veamos el ejercicio páginas dinámicas en el servidor a modo de prueba de evidencias**

**Figura 5:** Página Dinámica - Ejemplo

### 3.6.5 Páginas dinámicas con acceso a una base de datos

Un servidor de aplicaciones nos permite trabajar con recursos del lado del servidor, como las bases de datos. Por ejemplo, una página dinámica puede indicar al servidor de aplicaciones que extraiga datos de una base de datos y los inserte en el código HTML de la página.

El uso de una base de datos para almacenar contenido permite separar el diseño del sitio Web del contenido que se desea mostrar a los usuarios del sitio. En lugar de escribir archivos HTML individuales para cada página, sólo se necesita escribir una página —o plantilla— para los distintos tipos de información que se desea presentar. Posteriormente, podremos cargar contenido en una base de datos y, seguidamente, hacer que el sitio Web recupere el contenido en respuesta a una solicitud del usuario.

La instrucción para extraer datos de una base de datos recibe el nombre de **consulta de base de datos**. Una consulta consta de criterios de búsqueda expresados en un lenguaje de base de datos denominado **SQL** (*Structured Query Language*, lenguaje de consulta estructurado). La consulta SQL se escribe en los scripts o etiquetas del lado del servidor de la página.

Un servidor de aplicaciones no se puede comunicar directamente con una base de datos porque el formato de esta última impide que se descifren los datos, de una forma bastante similar a cuando un documento de Microsoft Word no puede descifrarse al abrirlo con el Bloc de Notas. El servidor de aplicaciones sólo se puede comunicar con la base de datos a través de un **controlador** que actúe de intermediario con la base de datos: el software actúa entonces como un intérprete entre el servidor de aplicaciones y la base de datos.

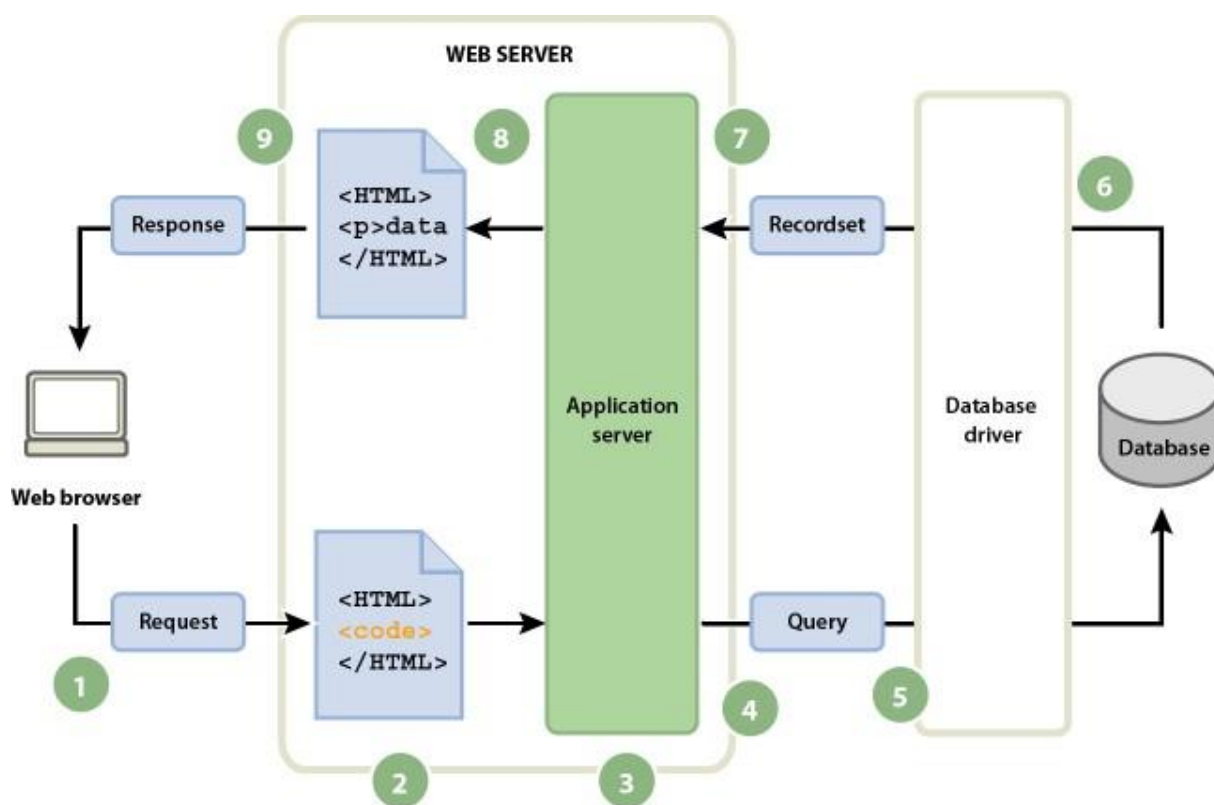
Una vez que el controlador establece la comunicación, la consulta se ejecuta en la base de datos y se crea un juego de registros. Un **juego de registros** es un conjunto de datos extraídos de una o varias tablas de una base de datos. El juego de registros se devuelve al servidor de aplicaciones, que emplea los datos para completar la página.

A continuación se ofrece una consulta de base de datos sencilla escrita en SQL:

```
SELECT NOMBRE, APELLIDOS, ASIGNATURA, NOTA  
FROM NOTAS_ALUMNOS
```

Esta instrucción crea un juego de registros de cuatro columnas y lo completa con filas que contienen el nombre, los apellidos, la asignatura y la nota de todos los alumnos de la base de datos.

En el siguiente ejemplo se muestra el proceso de consulta de base de datos y de devolución de los datos al navegador:



**Figura 6:** Página Dinámica con acceso a datos

1. El navegador web solicita la página dinámica. 2. El servidor web localiza la página y la envía al servidor de aplicaciones. 3. El servidor de aplicaciones busca instrucciones en la página. 4. El servidor de aplicaciones envía la consulta al controlador de la base de datos. 5. El controlador ejecuta la consulta en la base de datos. 6. El juego de registros se devuelve al controlador. 7. El controlador pasa el juego de registros al servidor de aplicaciones. 8. El servidor de aplicaciones inserta los datos en una página y luego pasa la página al servidor web. 9. El servidor Web envía la página finalizada al navegador solicitante.

### 3.6.6 Cliente estático o cliente dinámico

Desde el punto de vista del **cliente web** (generalmente el navegador) las páginas que componen una aplicación web también se pueden dividir en estáticas y dinámicas.

Las estáticas son aquellas en las que el contenido visualizado por el cliente no cambia, a menos que se realice un clic en un enlace y se cargue una nueva página. Las dinámicas son aquellas en las que el contenido mostrado por el cliente cambia sin necesidad de recargar la página. Estos cambios pueden utilizarse para producir efectos gráficos o realizando peticiones asíncronas de tal forma que el contenido de la página cambie sin necesidad de obligar a una recarga de esta.

Para crear páginas web de cliente dinámicas se usan técnicas de manipulación del *Document Object Model* ([DOM](#)) del navegador junto con peticiones asíncronas ([AJAX](#) - Asynchronous Javascript and XML). Esta técnica llevada al extremo se conoce como aplicación SPA ([Single Page Application](#)).

Hoy en día, la mayoría de las aplicaciones web se construyen con métodos dinámicos tanto desde el punto de vista del servidor como del cliente. Sólo hay que pensar en aplicaciones tipo Gmail, Facebook, Google Maps, Twitter, etc.

### *3.6.7 Cliente estático creado a partir de texto plano (Static Site Generator)*

Los generadores de sitios estáticos funcionan convirtiendo texto simple y con formato ligero (generalmente [markdown](#)) en sitios web o blogs estáticos. No hay bases de datos para ralentizar las solicitudes y el sitio es más fácil de mantener, tiene opciones de seguridad mejoradas y, sin embargo, tiene una infraestructura menor en coste.

Por ejemplo: [Jekyll](#), [Hexo](#), [Hugo](#), [Pelican](#), [Middleman](#), [Metalsmith](#), [Ghost](#).

También entra dentro de esta categoría las páginas creadas a partir de [GitHub Pages](#).

Los generadores de sitios estáticos (**SSG**) hacen lo mismo que una web dinámica, pero sin las desventajas de ésta. Aplican datos y contenido a las plantillas y generan una vista de una página que se puede mostrar a los visitantes de un sitio.

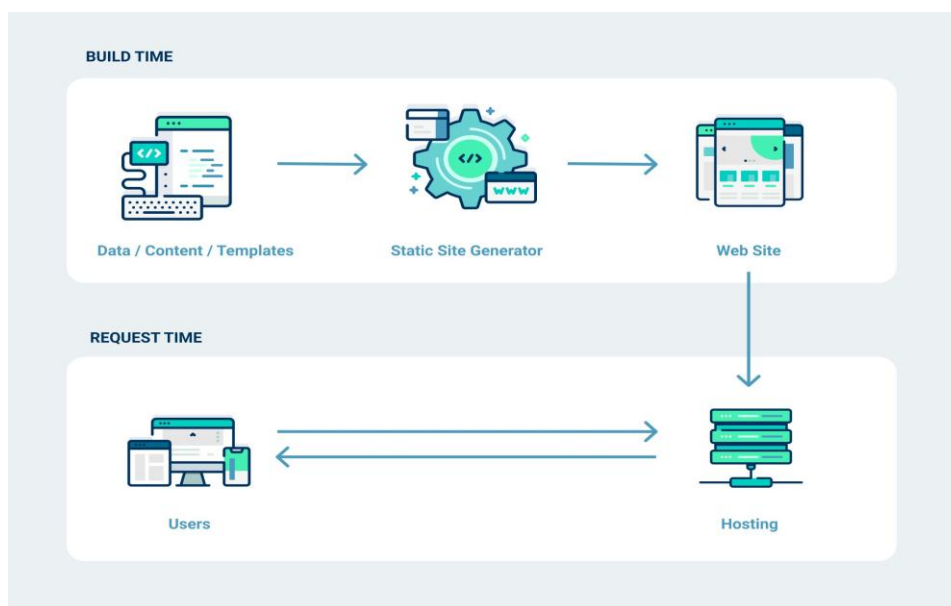
La mayor diferencia entre un generador de sitios estáticos y una pila de aplicaciones web tradicional es que, en lugar de esperar hasta que se solicite una página y luego generar su vista a petición cada vez, un generador de sitios estáticos hace esto con anticipación para que la vista esté lista para ser servida con anticipación. Y lo hace para cada vista posible de un sitio en el momento de la construcción.

#### **Tienen una ventaja muy importante desde el punto de vista de la seguridad:**

Dado que los generadores de sitios estáticos crean un conjunto de activos estáticos que pueden ser servidos desde un servidor web simplificado, o mejor aún, directa y completamente desde una red de distribución de contenido (CDN), tienen un perfil de seguridad notablemente bueno. Dado que se procesan con anticipación y están listos para servir, la infraestructura involucrada en servirlos puede simplificarse enormemente y tener muy pocos vectores de ataque malicioso. Cuando eliminamos la necesidad de que los servidores realicen la lógica y el trabajo, eliminamos las formas en que los malos actores les inyecten código malicioso y los engañen para que realicen acciones nefastas.

Y cuando no necesitamos acceder a bases de datos, realizar operaciones lógicas o modificar recursos para cada vista, podemos simplificar drásticamente nuestra infraestructura de alojamiento. Esto también mejora aún más la seguridad, ya que físicamente hay menos servidores involucrados en la gestión de solicitudes.

No hay servidor más seguro que la ausencia de servidor. Cuanta menos infraestructura necesite nuestra aplicación mejor desde el punto de vista de la seguridad. Sólo debemos instalar y/o activar aquellos recursos necesarios para que nuestra aplicación funcione. Cuantos menos módulos debamos instalar, mejor para la seguridad ya que nos hemos de preocupar de menos vectores de ataque.



**Figura 7:** Flujo en SSG

Una de las **mayores plataformas para desplegar** Static Site Generators es <https://www.netlify.com/>. En el siguiente enlace encontrarás un libro gratuito de dicha plataforma con el título Modern Web Development on the JAMStack. Por cierto, JAM son las iniciales de **J**avascript, **A**PI's y **M**arkup

## 4 Tecnologías de desarrollo Web

Como hemos comentado en el apartado Aplicaciones Web, hoy en día se utilizan páginas dinámicas tanto del lado del servidor como del cliente.

El impacto de la Web ha propiciado la aparición de una gran cantidad de tecnologías, librerías, herramientas y estilos arquitectónicos para desarrollar una aplicación web. Para facilitar la tarea de escoger cuál es la tecnología más adecuada para un proyecto, es conveniente conocer los elementos más importantes desde un punto de vista de alto nivel para tener una visión global de la programación web.

Existen dos enfoques en el desarrollo de aplicaciones web:

- Creación de aplicaciones web con integración de tecnologías de desarrollo
- Creación de aplicaciones web con sistemas gestores de contenido

### 4.1 Integración de tecnologías de desarrollo

Una de las formas de crear una aplicación web es implementarla usando componentes ya existentes e integrándolos con tecnología propia para adecuarlos a la lógica de negocio. Una premisa importante en cualquier tipo de desarrollo es «*no reinventar la rueda*». Seguro que existe algún componente que ya hace lo que pretendemos hacer.

Dentro de las tecnologías de desarrollo, podemos distinguir entre:

- **Tecnologías de cliente:** Tecnologías que permiten crear interfaces de usuario atractivos y permiten la comunicación con el servidor. Basadas en

HTML, CSS y JavaScript. Entre las de propósito general, cabe destacar [jQuery](#), [AngularJS](#), [Bootstrap](#).

- **Tecnologías de servidor:** Tecnologías que permiten implementar el comportamiento de la aplicación web en el servidor: lógica de negocio, generación de informes, compartir información entre usuarios, envío de correos, etc.
  - En **PHP**, tenemos [CodeIgniter](#), [Symfony](#) y [Laravel](#).
  - En **Java**, [Spring MVC](#), [JSE](#), [Struts](#).
  - En **javascript**, tenemos [node.js](#) y el framework más utilizado es [Express](#).

En **Bases de datos**, tenemos Oracle, MySQL, Microsoft SQL Server, PostgreSQL

## 4.2 Sistemas gestores de contenido (CMS)

Existen aplicaciones web cuya principal funcionalidad es la publicación de contenido: blogs, páginas de empresas, organismos públicos, comercio electrónico, etc.

Todas estas webs tienen mucho en común, prácticamente sólo se diferencian en el contenido y en el aspecto gráfico.

Para desarrollar este tipo de webs, en vez de desarrollar la web con integración de tecnologías de desarrollo se usa una aplicación ya creada que se puede personalizar y adaptar (mayormente vía

web).

A las aplicaciones de este tipo se las denomina **Sistemas Gestores de Contenido** (*Content Management Systems*).

Entre las **ventajas** de contar con un CMS están:

1. **Fácilmente actualizable.** La edición del contenido está separada del diseño y funcionalidad del sitio. Esto permite introducir contenido a usuarios con poca formación técnica sin interferir en el diseño.
2. A cada usuario se le pueden asignar permisos de acceso selectivo basados en sus **roles** (por ejemplo, puede permitir que algunos usuarios solo agreguen y edite su propio contenido, mientras que otros le dan acceso universal).
3. **Uso de plantillas totalmente personalizables.** Tanto para el contenido como para el diseño de tu propio sitio web. Esta es una de las partes que más suelen asustar si no se tienen conocimientos. Pero ahora no hace falta ser un experto en diseño.
4. Los componentes básicos de la aplicación como menús, cabeceras, pies de página y laterales se mantienen desde la interfaz de administración.
5. Además, **los mejores CMS incluyen plugins** para poder sacar aún más partido a todas las funcionalidades

Evidentemente, no son todas ventajas. Los principales **inconvenientes** de usar un CMS son:

1. Desafortunadamente, como el código fuente está disponible al público en general y *todas las aplicaciones tienen bugs* de seguridad, hay algunos

- hackers malévolos por ahí que pueden averiguar cómo entrar en estas plataformas; por lo que la seguridad requerirá precauciones adicionales.
2. Hacer que el sitio web se vea exactamente como queremos puede suponer desafío. Esto es cierto en algunos *frameworks* de CMS más que otros, pero todos presentan un poco más de trabajo para definir un «estilo» propio.
  3. El CMS almacena todo por separado, luego lo monta en el momento en que el cliente web solicita una página, lo que significa que pueden ser lentos; sin embargo, esto se puede mitigar mediante el uso de un almacenamiento en [caché](#) fuerte y eficaz y con una [Red de Entrega de Contenidos](#) (*Content Delivery Network* - CDN).
  4. Limitaciones de funcionalidad: Hay algunas cosas que no se pueden hacer en un CMS, al menos no sin reescribir parte del código.

Los principales CMS *libres* propiamente dichos son [WordPress](#), [Drupal](#) y [Joomla](#). Para una estadística más detallada podemos consultar la página de tendencias de [buildwith.com](#)

Existen otros tipos de aplicaciones orientadas a otros ámbitos, como por ejemplo al comercio electrónico. Entre ellas cabe destacar [WooCommerce](#) y [Magento](#) que se utilizan para la creación de tiendas virtuales.

#### 4.2.1 Headless CMS

Ahora bien, en los últimos años hemos podido presenciar el surgimiento de una nueva clase de CMS, el «headless CMS». Ciertamente, el término traducido al español no resulta muy explicativo («CMS sin cabeza»), por lo que vamos a explicar un poco en qué consisten.

Este tipo de CMS sólo se encargan de generar una interfaz web para introducir los datos. Una vez creada esta interfaz y alimentada con datos, proveen de una API REST, generalmente en JSON, para consultarlos.

Después ya sólo hace falta conectar los datos con:

1. Un front-end para Web
2. Otro para móvil
3. Otro para relojes inteligentes.
4. ...